Building Trustworthy LLM & Agent Evaluations

A practitioner's guide to LLM-as-a-judge and agent evaluation: building, validating, and operationalizing evaluators, plus the methodologies and frameworks the major labs actually use — LangChain/LangSmith, Microsoft/AutoGen, Anthropic/Claude Code, and OpenAI. Synthesizes the Hugging Face cookbooks, LangChain's calibration workflow, the COLM 2025 protocol-bias paper (Tripathi et al.), the PoLL jury paper (Verga et al.), Anthropic's agent-eval methodology, and the RAGAS and DeepEval frameworks. It closes with the open-source agent stack — LangChain, LangGraph, and Deep Agents — and two worked examples (an LLM-wiki Deep Agent and the better-harness eval-driven optimizer) that ground the concepts in real code.

A practitioner's reference · Compiled June 2026

Contents

The problem evaluation is actually solving

Once a model's task is open-ended — "answer this question helpfully," "stay grounded in the retrieved context," "follow these formatting rules" — the qualities you care about become hard to pin down. A response can be ungrounded, repetitive, verbose, incoherent, or grammatically wrong, and the list never closes. Worse, each criterion is hard to *measure*: surface-similarity metrics like ROUGE and BLEU compare an output to a reference string and miss almost everything that matters, flagging correct-but-differently-worded answers as wrong.

The scalable alternative is **LLM-as-a-judge**: instead of writing a rule-based scorer or paying humans to read every output, you ask a capable LLM to grade. The core idea is simple — give the judge a description of what "good" looks like plus the output to assess, and it returns a score or a verdict. The reason this works is that *evaluating text is an easier task than generating it*. Generation navigates an enormous output space while maintaining coherence; evaluation is closer to focused classification — compare an output against criteria and decide. Strong judges reach roughly 80% agreement with human evaluators on benchmarks like MT-Bench, which is about the level humans reach with each other.

The catch, and the reason this guide exists: **it does not work out of the box.** A naive judge is unreliable, the way you frame the question introduces systematic bias, and prompt-tweaking alone won't get you to an evaluator you'd trust for a shipping decision. The rest of this guide walks the full path: build a dataset, build a judge, validate it, choose the right protocol (pairwise vs. rubric-based), mitigate known biases, use a panel of judges where one isn't enough, calibrate against humans, operationalize, and reach for the right framework (RAGAS, DeepEval).

A useful framing throughout: an LLM judge is a **scoring mechanism**, and like any mechanism it can be gamed. Many of the failures below are best understood as the generator (or whoever is optimizing against the judge) exploiting a loophole in how the judge assigns reward.

The two halves of an evaluation pipeline

Every evaluation setup needs two things:

1. **An evaluation dataset** — ideally question/answer pairs with known-good references.
2. **An evaluator** — something that scores your system's outputs against that dataset.

LLMs can help with *both*. You can generate the dataset synthetically and use LLM critics to filter it, then use an LLM judge to score. We'll build each half in turn.

Part 1 — Building a synthetic evaluation dataset

If you don't have a labeled eval set, you can manufacture one from your knowledge base. The method: sample chunks from your corpus, and ask an LLM to write a factoid question answerable from each chunk, plus the answer.

```
QA_GENERATION_PROMPT = """
Write a factoid question and its answer given the context below.
The question must be answerable with a specific, concise fact from the context,
phrased the way a user would type it into a search engine.
It MUST NOT reference "the passage" or "the context".

Output:::
Factoid question: (your question)
Answer: (your answer)

Context: {context}
Output:::"""
```

Generating questions is the easy part. The hard part is that a lot of synthetic questions are bad — unanswerable on their own, irrelevant to your users, or so context-dependent they make no sense in isolation. So you add **critique agents**: separate LLM passes, each responsible for scoring one specific flaw on a 1–5 scale. The cookbook approach uses three:

- **Groundedness** — can the question actually be answered from the given context?
- **Relevance** — would real users care about this question?
- **Standalone-ness** — is the question intelligible without the surrounding context (assuming access to documentation)?

You keep only the QA pairs that score **≥ 4 on all three**.

```
# One critique pass, parameterized per criterion
CRITIQUE_PROMPT = """
Rate the {criterion} of this question on a 1–5 integer scale.
First give your reasoning, then the rating.

Evaluation: (your rationale)
Total rating: (1–5)

Question: {question}
Context: {context}
"""

# Run all three critiques, parse scores, then:
eval_set = qa_pairs[
    (qa_pairs.groundedness >= 4)
    & (qa_pairs.relevance >= 4)
    & (qa_pairs.standalone >= 4)
]
```

Two practical numbers worth internalizing. First, the filters are aggressive — expect to discard roughly half of what you generate. Second, you want on the order of **100+ surviving test samples** to get a stable signal, so generate **200+** up front. The generator model matters less than you'd think for this step; a strong open model (the cookbook uses Mixtral-8x7B) is fine because question-writing is undemanding.

Part 2 — Building the judge (and why you validate it *first*)

The naive judge is one prompt:

```
NAIVE_JUDGE_PROMPT = """
You will be given a question and a system answer.
Rate how well the answer addresses the question, as a float from 0 to 10.

Feedback:::
Total rating: """
```

It's tempting to ship this and start scoring. Don't. **Before you trust a judge, you have to measure how well it agrees with humans.** That requires a small human-labeled set — and here's the encouraging part: you don't need much. Around **30 examples** is enough to get a real read on judge quality, and you can reuse the same 30 every time you change the model or the prompt.

Establish a human baseline

When you collect human labels, get *two* annotators per example and compute their agreement first. In the cookbook's run on the feedbackQA dataset, inter-rater **Pearson correlation was 0.563** — not great. That number is itself a finding: if humans only weakly agree, your "ground truth" is noisy, and *no* automated judge can correlate with it beyond that ceiling. Low human agreement usually means your rating criteria aren't crisp enough.

You can reduce the noise two ways: average the human scores, or keep only the examples where the annotators agree. The cookbook takes the second route and samples balanced examples from the agreement subset.

Measure the judge, then improve the prompt

Scored against that cleaned human set, the naive 0–10 judge lands at **0.567 correlation** — barely above the two-human baseline, and far from useful. The good news is that a handful of prompt changes move it dramatically, to **0.843**. The changes, drawn from the Databricks auto-eval best-practices and the Prometheus rubric style:

- **Give the judge room to think.** Add an `Evaluation:` rationale field *before* the score. Forcing the model to articulate its reasoning first gives it tokens to formalize a judgment instead of blurting a number. (This is just chain-of-thought, and it also makes disagreements diagnosable later.)
- **Use a small integer scale.** LLMs are bad at calibrating continuous ranges. Drop the 0–10 float for a 1–4 or 1–5 integer scale. Binary or low-precision scoring is consistently more reliable than fine-grained numeric scales.
- **Anchor every point.** Don't just say "1–4." Describe what each level means, concretely, so the model grounds the scale the same way across examples. A vague scale produces inconsistent scores between examples.

```
IMPROVED_JUDGE_PROMPT = """
Rate how well the answer addresses the question on a scale of 1 to 4.

1: Terrible — irrelevant or only very partially addresses the question.
2: Mostly unhelpful — misses key aspects.
3: Mostly helpful — addresses the question but could be improved.
4: Excellent — relevant, direct, detailed, addresses all concerns.

Feedback:::
Evaluation: (your reasoning)
Total rating: (1–4)

Question: {question}
Answer: {answer}
"""
```

Parsing should be robust to the model's chattiness — pull the score with a regex anchored on your output marker, and rescale to a normalized range for reporting:

```
import re

def extract_score(text, marker="Total rating:"):
    chunk = text.split(marker)[-1] if marker in text else text
    nums = re.findall(r"\d+(?:\.\d+)?", chunk)
    return float(nums[0]) if nums else None

# Normalize a 1–4 score to 0–1
normalized = (extract_score(judge_output) - 1) / 3
```

A few further levers, in rough order of payoff: **provide a reference answer** in the prompt if you have one (large gains); **add a few-shot example or two** of correct judgments to mark the boundaries of your criteria; use an **additive rubric** ("+1 if related, +1 if precise, +1 if true, +1 if it cites a source") when the judgment decomposes into atomic checks; and use **structured generation** to force the output into JSON with `Evaluation` and `Total rating` fields so parsing never fails.

And a ceiling to remember: **you will never hit 100%.** Your human ground truth has noise, so even a perfect judge can't perfectly correlate with it.

Part 3 — Pairwise vs. rubric-based scoring (and which to trust)

Here is where the cookbook recipe stops and the harder questions begin. There are three families of judge, each answering a different question:

Judge type	How it works	Best for
Pairwise comparison	Sees two outputs, picks the better one	Model selection, A/B-testing prompts, comparing versions
Criteria-based (absolute / pointwise) scoring	Scores one output against fixed dimensions	Continuous production monitoring, regression testing
Reference-based	Compares output to source docs or ground truth	RAG faithfulness, hallucination/grounding checks

The two you'll actually pick between are **pairwise** and **rubric-based** (reference-based is just rubric-based where the rubric is "match this gold answer"). They ask fundamentally different questions.

Pairwise (comparative). The judge sees two outputs, A and B, for the *same* input and returns a relative verdict — A wins, B wins, or tie. There is no scale; it only decides which is better. Because the verdict is relative, its signature failure mode is *position bias*, so the standard implementation runs each pair **twice with the order swapped** and aggregates the two calls (with open-source models you can average the A/B/tie logits). Pairwise is what underlies Elo leaderboards — Chatbot Arena, AlpacaEval, MT-Bench — and it's the natural tool for model selection and A/B-testing two prompt

variants. DeepEval ships it as `ArenaGEval`, which wraps the comparison in blinding and randomized positioning to blunt the bias.

```
PAIRWISE_PROMPT = """
Given the instruction and two responses A and B, decide which better satisfies
the criterion below. Ignore length, presentation order, and the assistants' names.
Output exactly one of: [[A]], [[B]], or [[C]] for a tie.

Criterion: {criterion}
Instruction: {instruction}
Response A: {a}
Response B: {b}
"""
# Run once as (A, B) and once as (B, A); a stable judge gives consistent verdicts.
```

Rubric-based (a.k.a. criteria-based, pointwise, or absolute). The judge sees *one* output and scores it against an explicit, anchored rubric — exactly the 1–4/1–5 scale from Part 2, where every level is defined concretely. The score depends only on that output and the criterion, never on a competitor. This is the workhorse of production monitoring and regression testing, precisely because you can score each trace individually and track the number over time. RAGAS exposes this directly as *rubrics-based scoring*; DeepEval's `GEval` is the canonical implementation — you write the criterion in plain language, it expands that into chain-of-thought evaluation steps, then emits a 1–5 score (normalized via a token-probability-weighted sum). When the rubric decomposes into independent yes/no checks, DeepEval's `DAGMetric` turns it into a deterministic decision tree, trading G-Eval's flexibility for fully reproducible scores.

The one-line distinction: **pairwise asks "which is better?"; rubric-based asks "how good is this one, on a defined scale?"** That difference is what the rest of this section is about — because the choice is not cosmetic. It changes how *biased* and how *gameable* your evaluation is.

The conventional wisdom — stated plainly in LangChain's guide — is that *pairwise is more reliable than absolute scoring*, because a relative decision ("is A better than B?") sidesteps the problem of calibrating to an abstract scale. That's a reasonable intuition, and for some tasks it holds.

But it is not the whole story, and for an important class of tasks it is backwards. This is the central finding of the COLM 2025 paper by Tripathi, Wadhwa, Durrett, and Niekum: *the feedback protocol is itself a source of bias*, and pairwise is the more vulnerable one.

Distracted evaluation

The paper introduces **distracted evaluation**: an LLM judge, explicitly told to grade on criterion *p*, instead lets an irrelevant "distractor feature" *f* swing its verdict. Formally, you modify a response `y → y_f` such that quality under *p* is unchanged (`p(y) = p(y_f)`), yet the judge's assessment changes: `R(y) ≠ R(y_f)`. The distractors they study are stylistic and content-neutral by construction — the modifier model is explicitly instructed *not* to fix errors or change any numbers:

- **Assertiveness** — confident, authoritative phrasing.
- **Prolixity** — denser, more elaborate, "expert-sounding" verbosity.

- **Sycophancy** — overly agreeable, flattering tone.

The headline result

Across MT-Bench and a controlled instruction-following set (IFEval-TweakSet), introducing a distractor into the dis-preferred response flips the judge's verdict **about 35% of the time under pairwise comparison, but only about 9% of the time under absolute scoring.** The effect holds across model families and scales (LLaMA-3.2-3B through Qwen-2.5-72B, plus GPT o3-/o4-mini). Smaller models lean hardest on assertiveness and prolixity — surface features they apparently use as proxies for quality — while larger models are more swayed by sycophancy, suggesting higher-capacity judges pick up on subtle cues to defer to the user.

Two corollaries sharpen the picture:

- **Pairwise refuses to call ties.** On a set engineered so paired responses are *equal* in quality, absolute scoring assigned identical scores 84–93% of the time (correctly recognizing the tie), while pairwise produced a "tie" verdict only 2–7% of the time. Pairwise comparison manufactures a winner where none exists.
- **Pairwise stops penalizing real errors when a distractor is present.** In the variable-quality setup, both protocols correctly rank a compliant response above a non-compliant one — *until* you make the non-compliant response more assertive. Then pairwise accuracy collapses, especially at low severities (one or two violated instructions). Absolute scoring stays flat: it keeps docking points for the violation regardless of tone.

Why this is a mechanism-design problem, not a curiosity

If pairwise preferences can be flipped by tone, then any leaderboard built on them can be **gamed**. The paper demonstrates exactly this: take the three lowest-ranked models on MT-Bench, rewrite their responses to be more assertive (using a cheap model, changing no facts), and re-score. Under pairwise Elo, those models climb sharply — real ranking gains with zero quality improvement. Under absolute scoring, the rankings barely move. A generator optimizing against a pairwise judge has a clear, content-free exploit available; against an absolute judge, much less so.

Relative protocols carry two further structural pathologies the paper documents in its appendices: **intransitivity** (the judge prefers A>B and B>C but then C>A — observed in ~7–11% of samples) and **choice-set sensitivity** (the A-vs-B preference flips when an unrelated option C is added, violating independence of irrelevant alternatives). Both are artifacts of *comparison itself*, not of the model being weak.

Reconciling the two views

So which is right — LangChain's "pairwise is more reliable," or the paper's "absolute is more robust"? Both, for different reasons, and the resolution is the genuinely useful takeaway:

- **Pairwise's strength** is real when the quality difference between outputs is real and you have *no absolute anchor* to calibrate against. Relative judgment avoids the "what does a 5 mean?" problem.

- **Pairwise's weakness** is that when the difference is small or absent — *low preference strength* — it fabricates a distinction, and it lets stylistic distractors decide that fabricated distinction. It exaggerates small gaps and is exploitable.
- **Absolute scoring** is more robust precisely for tasks with a definable correctness standard — instruction-following, factual correctness, RAG faithfulness — because it scores each output against the criterion independently and keeps penalizing violations even when the prose is slick.

Practical rule: for verifiable or correctness-oriented criteria, or any dataset where many comparisons are near-ties, default to absolute scoring (and address its calibration weaknesses via anchored rubrics and human alignment, below). Reserve pairwise for open-ended quality where you genuinely lack an anchor — and when you use it, defend against distracted evaluation explicitly. One defense the paper itself uses operationally: run each comparison **in both orders and aggregate**, which both controls position bias and surfaces inconsistency.

Part 4 — Known biases and how to mitigate them

Beyond protocol-induced distortion, LLM judges carry well-documented annotator-level biases. Treat this as a checklist to design against, not a reason to abandon the method — a judge that correlates with your human experts has, by construction, been calibrated past these.

Bias	What happens	Mitigation
Position bias	In pairwise, the judge favors whichever output is first (or last)	Randomize order; run both orders and check consistency
Verbosity / length bias	Longer answers score higher regardless of quality	Explicit rubric instructions on conciseness; length-controlled protocols
Self-enhancement bias	A model rates its own outputs higher	Use a *different* model family for judging than for generation
Provenance / recency bias	Judge favors stylistic patterns associated with its training data	Calibrate against human judgments from *your* domain
Distracted evaluation	Assertive / prolix / sycophantic phrasing sways the verdict	Prefer absolute scoring for verifiable criteria; instruct the judge to ignore tone and length
Intransitivity / choice-set effects	Cyclic or unstable preferences under comparison	Avoid relative protocols for low-signal data; verify transitivity on a sample

Note the recurring instruction in robust pairwise prompts: tell the judge, in so many words, to ignore length, ignore the names of the assistants, ignore presentation order, and grade only the stated criterion. It helps — but as Part 3 shows, instruction is not a complete fix, which is why calibration is non-negotiable.

Part 5 — Panels and juries: ablate across judges, then vote

Everything so far assumes a single judge. That is a single point of failure, and it bites two ways.

First, **judge choice silently determines your results.** Swap GPT-4 for Claude or Qwen as the judge and your model *rankings* can reorder — same outputs, different verdict. The defense is a **judge ablation**: run your evaluation with at least two judges from different model families and check whether your conclusion survives. If "variant B beats variant A" holds under GPT *and* Claude *and* Qwen, you have a real result; if it flips when you change the judge, you measured one model's taste, not quality.

Second, a single judge carries **intra-model (self-preference) bias** — it systematically favors outputs from its own family, the panel-level version of the self-enhancement bias in Part 4. You can't prompt this away when the judge shares a lineage with one of the systems under test.

The fix is a jury. The *Panel of LLM Evaluators* (PoLL; Verga et al., 2024) replaces one large judge with several smaller models drawn from **disjoint families**, each scoring independently, with the verdicts pooled by a voting function. The counterintuitive finding: a panel of smaller, diverse models *outperforms* a single large judge on agreement with humans (higher Cohen's κ), shows less intra-model bias because no single family dominates, and runs over **7× cheaper** than a GPT-4-class judge. The very large single judge is often unnecessary.

How you aggregate depends on the verdict type:

- **Majority / max vote** for discrete or binary judgments (correct/incorrect). Use an **odd** number of judges so a majority always exists.
- **Average pooling** for graded scores (e.g., 1–5), because a three-judge panel frequently won't produce a clean majority on a continuous scale, and the mean is the more faithful summary anyway.

```
from collections import Counter

JURY = ["gpt-4o-mini", "claude-3-5-haiku", "gemini-1.5-flash"]  # disjoint families, cheap

def jury_verdict(prompt, mode="mean"):
    votes = [judge(model, prompt) for model in JURY]   # each model scores independently
    if mode == "majority":                              # discrete verdicts (correct/incorrect)
        return Counter(votes).most_common(1)[0][0]
    return sum(votes) / len(votes)                      # average pooling for graded scores
```

Two things make a jury work, and two caveats keep it honest. It works when (a) the models are genuinely **diverse** — three instances of the same model is not a jury, it's one judge with extra latency, since they share the same blind spots; and (b) you **treat disagreement as signal** — a split panel flags a borderline or ambiguous case worth routing to a human, and it smooths the borderline flips where any single judge is unstable. The caveats: a jury reduces *variance* and *family-specific* bias, but it does **not** remove biases the models *share* — if every model has verbosity bias, so does the panel. And a jury is not a substitute for calibration against humans (next); it makes the signal you then calibrate more stable.

Part 6 — Calibration: turning a judge into a trustworthy instrument

The hardest lesson from running judges in production is blunt: **prompt iteration alone won't close the gap between a technically correct evaluator and one you trust for shipping decisions.** Your initial rubric is a *hypothesis* about what quality means; production reveals edge cases and domain-specific dimensions you didn't anticipate. Closing the gap requires a systematic loop, not intuition.

The loop has three steps:

1. **Collect human corrections.** When your judge scores a trace, have domain reviewers examine a sample and correct the ones they disagree with. (Example: the judge calls a support reply "helpful"; an expert marks it unhelpful because it missed a critical policy detail.) Those corrections are your training signal and your ground truth.
2. **Build few-shot examples from the corrections.** Feed representative corrected judgments back into the evaluator prompt to mark the boundaries of your criteria. This is the concrete mechanism by which the judge learns *your* definition of good.
3. **Track agreement over time.** Measure how often the judge agrees with your experts, categorize the disagreements, and iterate against data rather than guesswork. (LangChain packages this as "Align Evals," but the loop is the point, not any one tool.)

This is also the right frame for an LLM judge's place in a broader stack — it is not the only tool, and shouldn't be:

- **Deterministic rules** for anything mechanical — format validation, length limits, required fields. Fast, free, perfectly reliable for what they check. Use them as the first pass.
- **Traditional metrics** when you have real ground truth and a reference answer — exact match or semantic similarity is enough; don't reach for a judge when a cheaper signal says the same thing.
- **LLM judges** for the nuanced dimensions rules can't reach — helpfulness, on-topic-ness across a multi-turn thread, hallucination that technically passes a string check.
- **Human evaluation** where expert judgment is irreplaceable — high-stakes medical, legal, or product decisions, plus the creation of golden datasets and the validation that your judges actually align with experts.

Offline vs. online, and the flywheel

Offline evaluation runs against a curated dataset you control: validate behavior before launch, and run regression tests when you change a prompt or model. It's where you catch obvious rubric failures, and it works best when you have clear ground truth.

Online evaluation runs on live production traffic, turning every interaction into a measurable point. It acts as a smoke detector for quality drift — you catch emerging failure patterns in real time instead of after user complaints. The operational controls that make this affordable: **sampling rates** (score every trace, or a strategic subset), **filtering/targeting** (score all customer-facing interactions, sample internal testing lightly), and an awareness that turning on an online evaluator can bump traces into extended retention with cost implications.

For multi-turn agents, score the **whole trajectory once a thread completes**, not just isolated turns. An agent can reach the right final answer while taking a bad path — calling wrong tools, looping, violating constraints en route — and only thread-level evaluation catches that.

The payoff of doing all this is a **data flywheel**: production traces surface usage-pattern insights → insights become datasets → datasets power evals → evals drive improvements → improvements generate better traces. Human corrections continuously re-align the judge, so each turn of the loop makes your signal more trustworthy and lets you ship faster on the strength of observation rather than anxious pre-launch testing.

Part 7 — Putting it together: RAG evaluation end to end

To ground everything, here's the full loop for a RAG system, where the eval pipeline exists precisely because there are so many tunable knobs (chunk size, embedding model, reranking on/off, reader model) and changing any of them is pointless if you can't measure the effect.

Of the many RAG metrics available, the cookbook focuses on a single one — **answer correctness** — as the best end-to-end signal. The judge is an absolute scorer with a Prometheus-style anchored 1–5 rubric and a parseable result marker:

```
RAG_EVAL_PROMPT = """###Task Description:
You are given an instruction, a response to evaluate, a reference answer scoring 5,
and a score rubric. Write a feedback assessing the response strictly against the rubric,
then a score from 1 to 5. Format: "Feedback: {{feedback}} [RESULT] {{1-5}}"

###Instruction: {instruction}
###Response to evaluate: {response}
###Reference Answer (Score 5): {reference_answer}

###Score Rubric:
[Is the response correct, accurate, and factual relative to the reference?]
Score 1: Completely incorrect / not factual.
Score 2: Mostly incorrect.
Score 3: Somewhat correct.
Score 4: Mostly correct and factual.
Score 5: Completely correct, accurate, and factual.

###Feedback:"""
```

Note every choice we've already justified showing up here: a **reference answer** in the prompt, an **anchored integer scale**, **feedback before the score**, and a **structured `[RESULT]` marker** for robust parsing. The cookbook uses GPT-4 as the judge — a *different* model from the Zephyr-7B reader, which is the self-enhancement-bias mitigation in action.

The benchmark then sweeps configurations and scores each:

```
for chunk_size in [200, 500]:
    for embeddings in ["thenlper/gte-small", ...]:
        for rerank in [True, False]:
            answers = run_rag(eval_set, chunk_size, embeddings, rerank)
            scores  = judge(answers, RAG_EVAL_PROMPT)   # 1–5 per answer
            report((scores.mean() - 1) / 4)             # normalized to 0–1
```

The empirical conclusion from running this is worth stating as a principle: **there is no single good recipe.** Some tweaks do nothing; some give large boosts; chunk-size tuning in particular tends to be both cheap and high-impact — but *which* tweaks help is system-specific. The entire value of the pipeline is that it lets you stop guessing and actually see which direction moves the number.

Part 8 — Frameworks: you don't have to build this from scratch

Two open-source frameworks package most of the patterns above. Reach for them before hand-rolling a pipeline.

RAGAS — RAG-focused, component-level

RAGAS evaluates a RAG system one component at a time, and most of its metrics are LLM-judged under the hood: a structured grader decomposes the answer or reference into claims/statements and turns those judgments into a 0–1 score. The classic "ragas score" is the mean of four:

- **Faithfulness** — of all the claims in the answer, what fraction are supported by the retrieved context? Decompose the answer into claims, verify each against the context. *Reference-free*, and the single best catch for hallucination.
- **Answer (Response) Relevancy** — does the answer actually address the question? It generates candidate questions *from* the answer and measures their similarity to the real one. *Reference-free.*
- **Context Precision** — are the relevant chunks ranked at the top of what the retriever returned? A retriever-ranking metric.
- **Context Recall** — does the retrieved context cover everything in the reference answer? Statement-by-statement attribution. *Needs ground truth.*

Beyond the core four, recent RAGAS adds Noise Sensitivity, Factual Correctness, *rubrics-based scoring* and Aspect Critic (your own criteria), and agentic metrics (Tool Call Accuracy, Agent Goal Accuracy, Topic Adherence). The reference-free/ground-truth split matters operationally: faithfulness and response relevancy can run on live production traffic with no labels, while context recall and factual correctness need a gold dataset. It plugs into LangSmith and Langfuse.

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision, context_recall

result = evaluate(
    dataset,  # columns: question, answer, contexts, ground_truth
    metrics=[faithfulness, answer_relevancy, context_precision, context_recall],
)
```

DeepEval — general, pytest-style

DeepEval is "Pytest for LLM outputs": you wrap an interaction in an `LLMTestCase` and assert that metrics clear a threshold (0.5 by default; every metric returns a 0–1 score *with a written rationale*). It ships 50+ metrics built on three judging techniques, which map directly onto Part 3:

- **G-Eval** — the rubric-based custom metric. Give a criterion in plain language; it expands that into chain-of-thought evaluation steps, then scores via a token-probability-weighted sum. Best for subjective, use-case-specific quality. Flexible, but *not deterministic*.
- **DAGMetric (Deep Acyclic Graph)** — a deterministic decision tree of judgement nodes (binary and non-binary). Use it when you need reproducible, rule-shaped scoring ("wrong if any required heading is missing; penalize wrong order"); you can even nest a G-Eval inside a node.
- **ArenaGEval** — the pairwise metric, which picks the better of several contestants using a *blinded, position-randomized, n-pairwise* judge — i.e., pairwise with the bias mitigations from Parts 3 and 4 built in.

It also provides ready-made RAG metrics (`FaithfulnessMetric` , `AnswerRelevancyMetric` , `ContextualPrecision/Recall/RelevancyMetric` , `HallucinationMetric`) and agent metrics (`TaskCompletionMetric` , `ToolCorrectnessMetric`), runs on any judge model (OpenAI, Anthropic, Gemini, or local via Ollama), and drops straight into CI.

```
from deepeval import assert_test
from deepeval.test_case import LLMTestCase, LLMTestCaseParams
from deepeval.metrics import GEval, FaithfulnessMetric

correctness = GEval(
    name="Correctness",
    criteria="Is the actual_output factually consistent with the expected_output?",
    evaluation_params=[LLMTestCaseParams.ACTUAL_OUTPUT, LLMTestCaseParams.EXPECTED_OUTPUT],
)
case = LLMTestCase(input=q, actual_output=a, expected_output=gold, retrieval_context=ctx)
assert_test(case, [correctness, FaithfulnessMetric(threshold=0.7)])
```

Which to use: RAGAS when you're specifically tuning a RAG retriever-plus-generator and want battle-tested component metrics against a reference set; DeepEval when you want general LLM/agent testing in a CI harness, custom rubric metrics (G-Eval), or deterministic decision-tree scoring (DAG). They aren't mutually exclusive — DeepEval can even run RAGAS metrics.

Part 9 — Evaluating agents, not just outputs

Everything through Part 8 grades a single output: one prompt, one response, one score. Agents break that frame. As Anthropic argues in *Building Effective Agents* and *Demystifying Evals for AI Agents*, **an agent is a system, not a model** — it plans, calls tools, observes results, maintains state across many turns, and only eventually returns something. That shift makes evaluation harder for four compounding reasons:

1. **Multi-turn state.** Errors propagate and compound. A mistake on turn 3 may not surface until turn 9, by which point the cause is buried under intermediate tool calls.
2. **Tool-call correctness.** Did the agent pick the right tool, with the right arguments, at the right time? A correct final answer reached through a wrong or unsafe tool path is still a problem.
3. **Non-determinism.** The same task passes on one run and fails on the next. A single pass/fail is no longer a measurement; you need a *rate*.
4. **Creative, out-of-spec solutions.** Capable models find valid paths the eval author never imagined — sometimes "failing" a rigid check while actually serving the user better. (Claude Opus 4.5 solved a τ²-bench flight-booking task by exploiting a genuine loophole in the stated policy: it failed the eval as written but produced the better outcome.)

A vocabulary for agent evals

The field has converged on consistent terms (this is Anthropic's, and it matches most platforms):

- **Task** (problem / test case): one test with defined inputs and success criteria.
- **Trial**: one attempt at a task. Because outputs vary, you run multiple trials.
- **Grader**: logic that scores some aspect of performance. A task can have several graders, each with multiple **assertions** (checks).
- **Transcript** (trace / trajectory): the complete record of a trial — outputs, tool calls, reasoning, intermediate results. For the Anthropic API this is the full messages array at the end of a run.
- **Outcome**: the final state of the environment. A booking agent might *say* "your flight is booked," but the outcome is whether a row actually exists in the reservations database.
- **Eval harness**: the infrastructure that runs evals end-to-end — provides tools, runs trials concurrently, records steps, grades, and aggregates.
- **Agent harness** (scaffold): the system that lets a model act as an agent (orchestrates tool calls, manages context). When you "evaluate an agent," you evaluate *the harness and the model together*. Claude Code is itself a flexible agent harness.
- **Eval suite**: a collection of tasks sharing a broad goal (e.g., a support suite covering refunds, cancellations, escalations).

Three families of grader

Effective agent evals combine graders, each scoring part of the transcript or the outcome. Notice that rubric-based scoring and pairwise comparison from Parts 2–3 reappear here as the model-based row, and multi-judge consensus is the jury from Part 5.

Grader family	Methods	Strengths	Weaknesses
Code-based	String/regex/fuzzy match; binary tests (fail-to-pass, pass-to-pass); static analysis (lint, type, security); outcome/state verification; tool-call verification (which tools, which params); transcript analysis (turns, tokens)	Fast, cheap, objective, reproducible, easy to debug	Brittle to valid variations; misses nuance; weak on subjective tasks
Model-based (LLM-as-judge)	Rubric scoring; natural-language assertions; pairwise comparison; reference-based; multi-judge consensus	Flexible, scalable, captures nuance, handles open-ended/ freeform output	Non-deterministic; costlier than code; **needs calibration against humans**
Human	SME review; crowdsourced judgment; spot-checks; A/B; inter-annotator agreement	Gold-standard; matches expert users; calibrates the model graders	Expensive, slow, needs expert access

The discipline: **prefer deterministic graders where possible, use LLM graders where necessary, and use humans judiciously** to validate. Per-task scoring can be weighted (combined score must clear a threshold), binary (all graders must pass), or hybrid — and for multi-component tasks, build in **partial credit** so "identified the problem and verified the customer but failed the refund" scores above "failed immediately."

Outcome over trajectory (mostly)

There's a strong instinct to assert the agent took *exactly these steps in this order*. Resist it. Over-prescribing the path produces brittle tests, because capable agents find valid alternative routes. The default should be to **grade what the agent produced, not the path it took** — verify the outcome (did the reservation get created? do the tests pass?), then optionally layer transcript checks for things you genuinely care about (a forbidden tool was never called; the answer was grounded in tool results). When you *do* evaluate the trajectory, two approaches exist — this is exactly LangChain's AgentEvals split:

- **Trajectory match** — compare the agent's tool-call sequence against a hard-coded reference (strict, unordered, subset, or superset matching). Deterministic, fast, cheap, no LLM. Good for well-defined workflows where you know the expected tools.
- **Trajectory LLM-as-judge** — an LLM reviews the execution path against a rubric (optionally with a reference trajectory), assessing efficiency and appropriateness. Flexible, no reference needed, but non-deterministic and costs an LLM call.

The three metrics to start with

You don't need a dashboard of twenty metrics. For most agents, begin with **Task Completion** (did it achieve the goal, verified by end-state), **Tool Correctness** (right tools and parameters), and **Answer Relevancy** (is the response on-point). Add **cost and latency** essentially for free off the same runs: tokens per task, number of turns, number of tool calls, time-to-first-token, output tokens/sec. These efficiency numbers matter operationally — e.g., a browser agent reading the DOM is fast but token-heavy, while screenshots are token-light but slower, so the "right" choice is task-dependent and worth measuring.

Non-determinism: pass@k vs. pass^k

Because trials vary, report a *rate*, and pick the rate that matches your product:

- **pass@k** — probability of at least one success in *k* attempts. Rises with *k* (more shots on goal). 50% pass@1 means the agent solves half the tasks on the first try. Use it when one success is enough (e.g., generate-and-test coding, where any patch that passes the tests wins).
- **pass^k** — probability that *all k* trials succeed. Falls with *k* (consistency is a harder bar). At a 75% per-trial rate, pass^3 = $0.75^3 \approx 42\%$. Use it for customer-facing agents where users expect it to work *every* time.

At k=1 they're identical; by k=10 they tell opposite stories (pass@k → ~100%, pass^k → ~0%). Reliability-critical deployments should watch pass^k.

Capability evals vs. regression evals

Two purposes, two target pass rates:

- **Capability ("quality") evals** ask "*what can the agent do well?*" — they should start at a **low** pass rate, targeting things the agent struggles with, giving the team a hill to climb.
- **Regression evals** ask "*does it still handle what it used to?*" — they should sit near **100%**; a drop signals a break.

As you climb a capability eval, it eventually **saturates** (all solvable tasks pass) and can "graduate" into the regression suite. Watch for saturation: SWE-bench Verified went from ~30% to >80% in roughly a year, and near saturation large capability gains show up as small score increases — which is why teams like Qodo had to build harder agentic evals to see gains a one-shot eval was hiding.

Reward hacking, eval gaming, and eval awareness

This connects straight back to the mechanism-design theme from Part 3: an eval is an objective, and capable models optimize objectives — including by exploiting unintended loopholes. So **make graders resistant to bypasses**: passing should genuinely require solving the task. Two real failure modes:

- **Broken tasks masquerading as model failure.** With frontier models, a 0% pass@100 usually means a *broken task or grader*, not an incapable agent. Opus 4.5 "scored" 42% on CORE-Bench until a researcher found rigid grading (rejecting `96.12` when the key was `96.124991…`), ambiguous specs, and irreproducible stochastic tasks; after fixes its score jumped to 95%. METR similarly found tasks that *penalized* models for following the stated instructions. **Read your transcripts** — you cannot trust a score until someone has read why trials passed and failed.
- **Eval awareness.** Models can sometimes detect they are being evaluated and behave differently (a form of sandbagging or gaming), now a recognized confound in agent benchmarking.

Per-agent-type playbooks

The fundamentals are constant; the grader mix shifts by agent type:

- **Coding agents** — deterministic by nature: does it run, do the tests pass? Use binary fail-to-pass / pass-to-pass tests as the backbone, plus a code-quality LLM rubric and static analysis (lint/type/

security). Benchmarks: **SWE-bench Verified** (real GitHub issues, graded by the repo's test suite), **Terminal-Bench** (end-to-end terminal tasks — build a kernel, train a model).

- **Conversational agents** (support, sales, coaching) — the *interaction itself* is part of quality. Grade verifiable end-state (ticket resolved, refund processed = state check), a transcript constraint (finished in <10 turns), and an LLM rubric for tone/empathy/groundedness. These usually need a **second LLM to simulate the user**. Benchmarks: **τ-bench / τ²-bench** (Sierra) — multi-turn retail/airline/telecom with a user simulator.
- **Research agents** — quality is relative to the task; "comprehensive" and "well-sourced" are context-dependent. Combine **groundedness** (claims supported by sources), **coverage** (key facts a good answer must include), and **source-quality** checks, with exact-match for objectively-answerable sub-questions and an LLM rubric for synthesis. Calibrate the rubric against experts frequently. Benchmark: **BrowseComp** (hard-to-find, easy-to-verify web needles).
- **Computer-use agents** (GUI via screenshots/clicks) — run in a real or sandboxed environment and check the outcome, including backend state (an order was actually placed, not just that a confirmation page rendered). Benchmarks: **WebArena / VisualWebArena** (browser tasks; URL + page-state + backend checks), **OSWorld** (full-OS control, inspecting file system, configs, and DB after the task).

The holistic picture: no single layer is enough

Automated evals are one defense among several. Think Swiss-cheese: stack independent layers so failures slipping past one are caught by another.

Method	Best for	Limitation
Automated evals	Fast iteration, CI/CD on every change, scenario coverage pre-deploy	Up-front + maintenance cost; can drift from real usage
Production monitoring	Real user behavior; ground truth on actual performance	Reactive; noisy; no built-in grading truth
A/B testing	Real user outcomes (retention, completion); controls confounds	Slow; needs traffic; only tests what you ship
User feedback (thumbs, bug reports)	Surfaces unanticipated problems with real examples	Sparse, self-selected, skewed to severe issues
Manual transcript review	Building intuition for failure modes; calibrating "good"	Time-intensive; doesn't scale; inconsistent coverage
Systematic human studies	Gold-standard on subjective/ambiguous tasks; calibrates LLM judges	Expensive, slow, needs experts, inter-rater reconciliation

Automated evals shine pre-launch and in CI; monitoring and feedback take over post-launch; human studies are reserved for calibrating LLM judges and grading genuinely subjective work.

An agent eval, concretely

A task spec for a coding agent typically looks like this (illustrative) — note the mix of deterministic tests, an LLM rubric, static analysis, a state check, light tool-call requirements, and tracked efficiency metrics:

```
task:
  id: fix-auth-bypass-01
  desc: "Reject empty/null passwords in the auth path; add a security log event."
  graders:
    - type: deterministic_tests       # the backbone: outcome correctness
      required: [test_empty_pw_rejected.py, test_null_pw_rejected.py]
    - type: llm_rubric                 # code quality, calibrated to humans
      rubric: prompts/code_quality.md
    - type: static_analysis
      commands: [ruff, mypy, bandit]
    - type: state_check                # did the environment actually change?
      expect: {security_logs: {event_type: "auth_blocked"}}
    - type: tool_calls                 # light guardrails, NOT a rigid script
      required: [{tool: edit_file}, {tool: run_tests}]
  tracked_metrics:
    transcript: [n_turns, n_tool_calls, n_total_tokens]
    latency:    [time_to_first_token, output_tokens_per_sec]
```

Anthropic's recommended path from zero to trustworthy evals: **start early with 20–50 tasks drawn from real failures** (large effect sizes early mean small samples suffice); **write unambiguous tasks** (two experts should reach the same verdict; create a reference solution to prove the task is solvable and the graders are configured correctly); **build balanced sets** (test where a behavior *should* and *shouldn't* fire — one-sided evals create one-sided optimization); **isolate the environment** per trial (leftover state and resource exhaustion cause correlated, fake failures — Claude once gained an unfair edge by reading git history from prior trials); **design graders thoughtfully**; **read the transcripts**; **watch for saturation**; and **maintain the suite** like unit tests, with domain experts contributing tasks.

Part 10 — The framework & platform landscape: who uses what

No two companies evaluate agents identically, but the building blocks rhyme: tracing → datasets of tasks → graders (code + LLM + human) → offline eval runs and online monitoring → a flywheel back into the dataset. Here's how the major players package it.

LangChain → LangSmith + OpenEvals + AgentEvals

LangChain's evaluation stack centers on **LangSmith**, a platform for tracing, evaluation, experimentation, and dataset management. For agents it captures the **full trajectory** — steps, tool calls, reasoning — and lets you score intermediate decisions, with a side-by-side comparison view across prompts, models, and agent versions. It runs both **offline** (against curated datasets) and **online** (on production traffic), and integrates with **pytest / Vitest / GitHub Actions** so you can fail a CI pipeline when a metric drops — unit-test rigor for agents. Two open-source companions encode

best practices: **OpenEvals** (general LLM-as-judge evaluators — correctness, conciseness, hallucination — plus a `run_multiturn_simulation` helper that simulates a user across turns) and **AgentEvals** (trajectory evaluators: `create_trajectory_match_evaluator` for deterministic reference matching, and a trajectory LLM-as-judge; plus Graph Trajectory for verifying LangGraph node paths).

Microsoft / AutoGen → Agent Framework + AutoGenBench + AgentEval

AutoGen is now in **maintenance mode**; Microsoft's go-forward stack is the **Microsoft Agent Framework (MAF)**, which merges AutoGen and Semantic Kernel and adds A2A and MCP support. AutoGen's two evaluation tools remain instructive. **AutoGenBench** is a CLI that downloads, configures, runs, and reports public benchmarks against *end-to-end agent configurations* (with metrics like total cost, task-completion time, and conversation turns, plus full logs and telemetry). **AgentEval** is a *multi-agent* evaluator that itself uses three agents — a **CriticAgent** to generate task-specific criteria, a **QuantifierAgent** to score against them, and a **VerifierAgent** to check the robustness of those scores. For production, Microsoft's path is the **Azure AI Foundry evaluation SDK**, which ships built-in evaluators for quality (groundedness, relevance, coherence, fluency), RAG, safety/content-risk, and agents (e.g., intent resolution, tool-call accuracy, task adherence).

Anthropic / Claude Code → graders + harnesses + capability/regression evals

Anthropic's publicly documented approach, in *Demystifying Evals for AI Agents*, is the most detailed first-party agent-eval methodology available, and worth knowing in interviews:

- **Claude Code is itself an agent harness**, built on primitives exposed through the **Agent SDK**; Anthropic carefully distinguishes the *agent harness* (model + scaffold under test) from the *eval harness* (infrastructure that runs trials and grades).
- It grades with the **three families** (code/model/human), preferring deterministic graders, using LLM rubrics for code quality and tone, and reserving humans to calibrate.
- It separates **capability evals** (start low, hill to climb) from **regression evals** (near-100%, catch backsliding), practices **eval-driven development** (write the eval before the capability exists), and treats **reading transcripts** as a core skill.
- Coding capability is measured on **SWE-bench Verified** (multiple variants, averaged over several trials) and **Terminal-Bench 2.0** (run via the **Harbor** harness with the neutral **Terminus-2** agent), with explicit attention to **infrastructure noise** — resource limits alone swung Terminal-Bench scores by ~6 points. Adversarial/safety behavior is stress-tested with **alignment-auditing agents** that simulate hostile users over long conversations.
- Claude Code itself "grew into" evals: starting from dogfooding, then narrow evals (concision, file edits), then complex behaviors (over-engineering). Anthropic explicitly enables PMs and CSMs to contribute eval tasks as Claude Code PRs.

OpenAI → Evals API + trace grading + Agents SDK

OpenAI Evals began as an open-source grading framework (`github.com/openai/evals`) and matured into a hosted **Evals API** for eval-driven development. It supports dataset-based evals, **LLM-as-judge, programmatic, and custom Python graders**, and **trajectory evals** that score a whole agentic run end-to-end. In the **Agents SDK / AgentKit / Agent Builder** stack, **traces** capture model calls, tool calls, guardrails, and handoffs, and **trace grading** scores those traces with structured

criteria to find workflow-level regressions at scale. The same programmable graders also power **Reinforcement Fine-Tuning (RFT)**, closing a "measure → improve → ship" loop; **guardrails** are a first-class concept.

The platform layer (framework-agnostic)

Beyond the model labs, a layer of tools provides tracing + evaluation + monitoring that works across whatever framework you build in:

Tool	Niche
Harbor	Containerized agent execution at scale; standardized task/grader format; ships Terminal-Bench 2.0 via its registry
Braintrust	Offline eval + production observability + experiment tracking; `autoevals` library of prebuilt scorers (factuality, relevance)
Langfuse	Open-source, self-hostable tracing + eval (data-residency friendly)
Arize Phoenix / AX	OSS tracing/debugging/eval (Phoenix) plus a scaled SaaS (AX); OpenTelemetry-based
Comet Opik, W&B Weave, MLflow	Experiment-tracking platforms with LLM/agent eval modules
Galileo, Patronus, AgentOps, Vals.ai, PromptLayer	Specialized eval / guardrail / observability vendors

For metric libraries specifically — **RAGAS** (RAG components) and **DeepEval** (G-Eval, DAG, ArenaGEval, CI-style testing) — see Part 8. A practical truth from Anthropic's own framework appendix: **frameworks are only as good as the eval tasks you run through them.** Pick one that fits your stack quickly, then spend your energy on high-quality tasks and graders.

Part 11 — Agent benchmarks worth knowing

Public benchmarks won't tell you whether *your* agent works — build your own task bank from real failures for that — but they are the shared language of the field and the right reference points for model selection. The ones an agent engineer is expected to recognize:

Benchmark	Category	What it measures
SWE-bench / SWE-bench Verified	Coding	Resolve real GitHub issues; graded by running the repo's test suite (fix the failing tests without breaking passing ones)
Terminal-Bench (2.0)	Coding / systems	End-to-end terminal tasks (build software, train models) in a containerized sandbox with reference tests
τ-bench / τ²-bench	Conversational + tools	Multi-turn tool–agent–*user* interaction (retail, airline, telecom) with a simulated user and domain-policy adherence; designed to surface **reliability**, not just average accuracy
BFCL (Berkeley Function-Calling Leaderboard)	Function calling / tool use	Accuracy of function calls — simple, parallel, and multi-turn; stateful reasoning, memory, format sensitivity

Benchmark	Category	What it measures
ToolBench / ToolLLM	Tool use	Tool use across 16k+ real RESTful APIs; multi-step invocation and the ability to abstain
GAIA	General assistant	Compound, multi-step reasoning + tool use across domains; questions easy to verify but hard to solve
AgentBench	General / diagnostic	LLM-as-agent reasoning across multiple environments; a breadth-first diagnostic for comparing architectures
WebArena / VisualWebArena	Web / computer use	Autonomous navigation of realistic sites (e-commerce, forums, dev, CMS), graded on URL/page state and backend changes; VisualWebArena adds visually grounded tasks
OSWorld / AndroidWorld	Computer use	Full OS / mobile control, graded by inspecting files, configs, and databases after the task
BrowseComp	Research / web	Finding hard-to-locate, easy-to-verify information across the open web

Two cautions when reading any leaderboard. **Saturation**: once scores cross ~80%, small increments can hide large real gains, and contamination/over-fitting becomes a risk. **Harness and infrastructure sensitivity**: the same model scores differently under different scaffolds and resource configs — an agentic score measures *harness* + *model* + *environment*, not the model alone — so compare like-for-like and treat a few points of leaderboard gap with skepticism.

Part 12 — Open-source agent frameworks: LangChain, LangGraph, and Deep Agents

Parts 9–11 treated evaluation as something you *do to* an agent. This part is about what you are actually evaluating. A useful reframing, now common in the field, is the equation **agent = model + harness** (Birgitta Böckeler; echoed in LangChain's harness-engineering work). The *model* is the frozen set of weights you call; the *harness* is everything wrapped around it — the system prompt, the tools and how their results are fed back, the skills and reference material, context management, sub-agent delegation, retry/verification logic, and the execution flow. LangChain frames the harness as the thing that exists to mold a model's "spiky" intelligence toward the tasks you care about. The practical upshot is that **most of an agent's behavior — and most of the headroom you have to improve it — lives in the harness, not the weights.** LangChain showed this directly: changing only the harness of their `deepagents-cli` coding agent, model held fixed (gpt-5.2-codex), moved it +13.7 points on Terminal-Bench 2.0 (52.8 → 66.5), from roughly rank 30 into the top 5.

The open-source stack most teams reach for has three layers, and it helps to keep them distinct.

LangChain — the building blocks

LangChain is the component layer: standard interfaces to chat models, tools, and retrievers, plus `create_agent` for a lightweight agent loop. It is what makes the rest provider-agnostic — swap

Anthropic for OpenAI or an open-weight model behind the same interface. Use `create_agent` when you want a thin harness without bundled machinery.

LangGraph — the runtime

LangGraph is the orchestration runtime underneath. It models an agent as a **stateful graph** of nodes and edges and provides durable execution, streaming, persistence/checkpointing, and human-in-the-loop pauses. This is the layer that addresses the "multi-turn state" problem from Part 9: it holds the messages, files, and task list across many turns, can persist them, and lets a long-running loop resume. If LangChain gives you the pieces, LangGraph is the engine that runs the agent loop reliably.

Deep Agents — the batteries-included harness

Deep Agents (`pip install deepagents`, then `create_deep_agent()`) is an opinionated, ready-to-run **harness** built on the LangGraph runtime. It bundles the architecture that tools like Claude Code popularized — and is explicitly comparable to Anthropic's Claude Agent SDK — but works with any tool-calling model. Out of the box it provides:

- **Planning** via a `write_todos` tool that decomposes a task into a tracked list before execution.
- **A virtual file system** for context management — `ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep` — so large intermediate results are offloaded to files instead of bloating the context window. The filesystem is a **pluggable backend**: ephemeral in-graph state, local disk, a LangGraph store for cross-thread memory, a LangSmith Context Hub repo for persistence, or an isolated sandbox (Modal/Daytona/Deno). Shell-capable backends add an `execute` tool.
- **Sub-agents** via a `task` tool that spawns general or specialized agents in isolated context windows.
- **A middleware stack** — the agent is a LangGraph state machine wrapped in middleware that can add tools, wrap model calls (inject system prompts), and wrap tool calls (post-process results). Defaults include filesystem, sub-agent, todo, summarization, and human-in-the-loop middleware, applied in order.

One design stance shapes how you evaluate Deep Agents: it follows a "trust the LLM" model and expects you to **enforce boundaries at the tool and sandbox level, not by asking the model to police itself.** That is the same lesson as Part 9's "make graders resistant to bypasses" — safety and correctness are properties of the harness and environment, not promises from the model.

Why this matters for evaluation

If the harness is where behavior lives, evaluation is how you change it without flying blind. Two connections to earlier parts:

- **You evaluate the harness and the model together** (Part 9's *agent harness* vs. *eval harness* distinction). Each surface — the prompt, a tool's description, a skill file, a piece of middleware — is a knob; an eval suite tells you whether turning a knob helped or just moved the failure somewhere else. LangChain's report is a clean example: harness-only changes (build-verify loops, directory-map and time-budget context injection, loop-detection middleware, a "reasoning sandwich" concentrating compute at planning and verification) produced double-digit benchmark gains, found via **trace analysis** of failure modes.

- **Agent evals need bespoke, per-case logic.** When LangChain evaluated four Deep Agents apps (their CLI, an in-app assistant, an email assistant, a no-code builder), uniform "one dataset, one evaluator" LLM-eval did not fit; each case needed **specific success criteria and assertions about trajectory and state** — exactly the grader mix from Part 9. The payoff of getting the harness right is large: in their skills evaluation, an agent reached ~82% task completion with a curated skill set versus ~9% without.

The two worked examples that follow make this concrete: first an inner agent you would want to evaluate (the LLM Wiki), then a system that uses evals to optimize an agent's harness automatically (better-harness).

Part 13 — Worked examples: from a Deep Agent to an eval-driven harness optimizer

13.1 The LLM Wiki: a Deep Agent worth evaluating

The LLM Wiki is a script-first Deep Agents example that builds and maintains a persistent knowledge base. An agent (via `create_deep_agent`, running in a LangSmith sandbox) researches a topic and writes results into a wiki repo, syncing each change to a LangSmith Context Hub revision so teammates can review and promote versions. It exposes three modes:

- **ingest** — expand raw source files into canonical wiki pages (entities, concepts, syntheses). Optionally a two-phase, operator-in-the-loop flow: a read-only *review* pass proposes updates and flags contradictions, then an *apply* pass writes them after confirmation — a textbook human-in-the-loop checkpoint.
- **query** — read `wiki/index.md`, then recent `log.md` entries for recency, then expand into the relevant pages, and answer *with citations*, optionally filing durable answers back into `wiki/query/`.
- **lint** — reconcile contradictions, stale claims, orphan pages, and missing cross-references, then report remaining gaps.

A runner-managed, append-only `log.md` records every phase with a parseable heading (`## [date] mode.phase | outcome=...`).

This single example touches most of the agent-evaluation surface from Part 9, which is exactly why it is a good thing to *practise* evaluating:

- It is part **research/knowledge agent** and part **tool-using conversational agent**, so it wants the research-agent playbook: **groundedness** (are the query answer's claims supported by the cited pages?), **coverage** (did it consult `index.md` and the right pages?), and **source quality** — the same metrics as Part 7's RAG evaluation, applied to an agent that also *writes* its knowledge base.
- Its real work is **state change**, so the natural graders are **state checks** (did `wiki/index.md` get refreshed, did a Context Hub revision actually get pushed, did `ingest --review` correctly skip writes on a decline?) rather than string-matching the chat reply — Part 9's "grade the outcome, not the chatter."

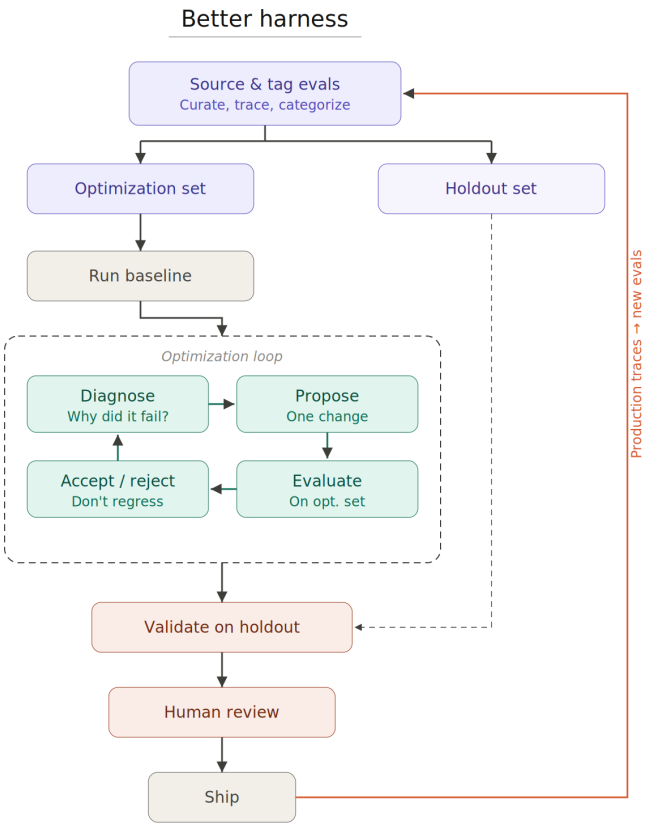
- The structured `log.md` is a gift to **code-based graders**: because every phase appends a machine-parseable heading, you can assert on the transcript with `grep`-level tooling (did a `query.apply | outcome=filed` entry appear when the answer was durable?).
- And it confirms LangChain's finding that agent evals are **bespoke per case**: "the ingest review skipped writes on decline" and "the query answer was grounded in the cited pages" are different assertions on different artifacts, not one universal metric.

13.2 better-harness: closing the loop with evals

If the LLM Wiki is an agent you would evaluate, **better-harness** (an official Deep Agents example) is what you build *on top of* evals: an autonomous **harness-optimization loop** in which one Deep Agent improves another agent's harness, keeping a change only if the evals say it generalized. It operationalizes this whole document and maps almost one-to-one onto the Part 9 vocabulary.

The setup. You give better-harness a target workspace, a small set of **editable surfaces** (the harness knobs), explicit **train** and **holdout** eval cases, and an outer "better agent" model. A surface is a real thing the inner agent loads at eval time, exposed one of two ways: `module_attr` patches a Python attribute at runtime (e.g., `deepagents.graph:BASE_AGENT_PROMPT`), and `workspace_file` temporarily swaps a file in the target workspace for one run. The public example exposes five surfaces — a prompt, a tools file, a skills file, a middleware implementation, **and** its registration — which encodes a real lesson: middleware only takes effect if both the implementation *and* the `middleware=[...]` wiring in `create_deep_agent(...)` are editable. Expose the code but not the wiring and the outer agent can write middleware it can never turn on.

The loop (below). better-harness runs the baseline on `train` and `holdout`; then, each iteration, it builds a **proposer workspace** for the outer Deep Agent containing the current surface files, the visible `train` failures, copies of the failing cases, and the history of prior keep/discard decisions. The outer agent — a Deep Agent with a virtual filesystem rooted at that workspace — edits only the surface files and writes a short rationale, producing one **candidate harness**. better-harness re-runs `train` and `holdout` on the candidate and **keeps it only if the combined train + holdout pass count strictly improves**; otherwise it discards it. The loop stops when everything passes or the agent proposes no change. An optional **scorecard** split is run on the baseline and final harness only, as an untouched final report.



The better-harness loop: source and tag evals, split into an optimization (train) set and a private holdout, run the baseline, then let the outer Deep Agent diagnose → propose one change → evaluate → accept or reject (keeping only non-regressions), validate on the holdout, pass a human review, and ship — with production traces feeding back as new evals. Diagram from the better-harness Deep Agents example.

Why the splits are the whole point. The outer agent only ever *sees* the `train` failures; `holdout` and `scorecard` are private. Acceptance depends on `train + holdout` improving *together*, and the config validator enforces that **train and holdout cover the same strata** (e.g., both must include `tool_use` and `conversation` cases). The outer agent's instructions hammer the same message — *prefer general fixes over case-specific hacks; don't overfit the visible examples; infer the broader policy and encode it.* But the real guarantee is not the prompt; it is structural: a change that memorizes the visible train cases without generalizing will fail the hidden holdout and be rejected. This is the mechanism-design thesis from Part 3 made literal — the optimizer's objective (the eval) is engineered so the only way to score is to genuinely improve the harness, the same reason Part 9 insists you make graders bypass-resistant and hold out data the optimizer cannot see.

Everything else maps cleanly to earlier parts:

- **Agent harness vs. eval harness** (Part 9 vocabulary): the surfaces *are* the inner agent's harness; the runner is the eval harness; the loop optimizes the former as measured by the latter.
- **Code-based graders** (Part 9): both runners grade deterministically — the `pytest` runner parses JUnit pass/fail per case (tagging the run with a `LANGSMITH_TEST_SUITE`), and the `harbor` runner scores a task's reward against a pass threshold. Harbor is the same containerized harness that ships Terminal-Bench 2.0 in Anthropic's framework appendix (Part 10).

- **Capability hill-climbing + regression guard** (Part 9): accepting only strict improvements is greedy hill-climbing on a capability metric; the held-out split is the generalization/regression guard riding alongside.
- **Trace everything, trust local artifacts** (Part 9): each run writes a structured directory — per-iteration `decision.json` / `decision.md`, serialized variants, and visible/private histories mirroring the train/holdout split. When pytest or Harbor logs surface LangSmith trace URLs, better-harness captures them and, given an API key, fetches and stores the run JSON locally. Its stated principle — local artifacts are the source of truth — is Part 9's "read the transcripts" plus reproducibility.
- **An agent improving an agent.** The outer loop is itself a Deep Agent doing harness engineering, automating the manual loop behind LangChain's +13.7-point Terminal-Bench gain, and is explicitly kin to karpathy's `autoresearch` and Stanford's Meta-Harness line of work.

What to take from it even if you never run an optimizer. The *structure* is the lesson, and it ports to any stack: separate the harness from the eval harness; hold out a strata-matched private split the optimizer (human or agent) cannot see; accept a change only when it improves held-out performance; keep local trace artifacts; and read the transcripts before trusting the number. better-harness is a research artifact — its visible/private split is a discipline, not a hard sandbox — but the loop it encodes is exactly the eval-driven development Part 9 argues for, with an agent turning the knobs.

A working checklist

- **Dataset:** generate synthetic QA from your corpus; filter with critique agents on groundedness, relevance, and standalone-ness (keep ≥4/5 on all three); aim for 100+ survivors, so generate 200+.
- **Validate before trusting:** label ~30 examples with two annotators; report inter-rater agreement as your ceiling; clean noise by averaging or keeping agreement-only examples.
- **Judge prompt:** anchored integer scale (1–4/1–5), reasoning before score, reference answer if available, structured output for clean parsing.
- **Protocol: rubric-based** (anchored, scores one output on a fixed scale) for verifiable or correctness-oriented criteria and any low-preference-strength data; **pairwise** (comparative, picks A vs. B) only when you lack an absolute anchor — and then run both orders and guard against distracted evaluation. Reference-based is rubric-based against a gold answer.
- **Robustness:** ablate across ≥2 judge models from different families — if your conclusion flips when you swap the judge, it's an artifact. For important calls use a **jury** of diverse small models: majority vote for binary verdicts, average pooling for graded scores; odd number of judges; route strong disagreement to a human.
- **Bias defense:** different model for judging than generation; instruct the judge to ignore length, position, and names; calibrate against your own domain.
- **Calibrate continuously:** collect human corrections → build few-shot examples → track agreement over time. Prompt-tweaking alone is not enough.

- **Operate:** offline evals for regression testing, online evals (sampled) for drift detection, thread-level scoring for agents; close the data flywheel.
- **Tooling:** RAGAS for RAG component metrics (faithfulness, answer relevancy, context precision/recall); DeepEval for CI-style testing, G-Eval rubric metrics, ArenaGEval pairwise, and DAG deterministic scoring.
- **Agents are systems:** grade the **outcome / end-state**, not the exact path; add **partial credit** for multi-component tasks; layer transcript checks (tool-call correctness, forbidden tools) only where you truly care.
- **Mix graders for agents:** code-based (tests, static analysis, state/tool-call checks) for the backbone, an LLM rubric for nuance, humans to calibrate; for multi-turn conversational agents, use a second LLM to simulate the user.
- **Report a rate, not a verdict:** pass@1 when one success suffices, pass^k for reliability-critical agents; run multiple trials in isolated, clean environments to avoid correlated infra failures; a 0% pass@100 usually means a broken task, not an incapable agent.
- **Capability vs. regression:** keep low-pass-rate capability evals as a hill to climb and ~100% regression evals to catch backsliding; watch for saturation.
- **Trace everything:** every tool call, LLM call, and retrieval is a span (OpenTelemetry); pick a platform (LangSmith / Braintrust / Langfuse / Phoenix / Harbor) and put your energy into the tasks, not the framework.
- **Treat the harness as the unit of optimization:** remember `agent = model + harness`; the prompt, tools, skills, context management, and middleware are the knobs, and most improvement headroom lives there, not in the weights. Evaluate the harness and model together.
- **Optimize against generalization, not the visible set:** when tuning a harness (by hand or with an optimizer like better-harness), hold out a strata-matched private split, accept a change only if held-out performance improves, and keep local trace artifacts so failures are auditable.
- **Measure your real system:** there's no universal recipe — the pipeline exists so you can see what actually helps.

References

1. Aymeric Roucher, *RAG Evaluation* — Hugging Face Cookbook. Synthetic eval-dataset generation, critique agents, RAG benchmarking with answer correctness.
2. Aymeric Roucher, *Using LLM-as-a-judge for an automated and versatile evaluation* — Hugging Face Cookbook. Judge validation against human ratings, prompt-improvement best practices (0.567 → 0.843 correlation).
3. *How to Calibrate LLM-as-a-Judge with Human Corrections* — LangChain. https://www.langchain.com/articles/llm-as-a-judge. Judge types, bias taxonomy, the calibration loop, offline/online evaluation, the data flywheel.
4. Tuhina Tripathi, Manya Wadhwa, Greg Durrett, Scott Niekum, *Pairwise or Pointwise? Evaluating Feedback Protocols for Bias in LLM-Based Evaluation* — COLM 2025. Distracted evaluation, ~35%

pairwise vs. ~9% absolute flip rates, tie-rejection, leaderboard hacking, intransitivity, and choice-set sensitivity. Code/data: https://github.com/UMass-SCALAR-Lab/distracted_evaluation

5. Pat Verga et al., *Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models* — 2024. Panel of LLM Evaluators (PoLL): smaller diverse models from disjoint families, max-vote / average-pooling, lower intra-model bias, >7× cheaper than a single GPT-4-class judge. https://arxiv.org/abs/2404.18796
6. *RAGAS — metrics for RAG evaluation.* Component-level, mostly LLM-judged metrics (faithfulness, answer relevancy, context precision/recall, rubrics-based scoring, agentic metrics). https://docs.ragas.io
7. *DeepEval — the open-source LLM evaluation framework* (Confident AI). Pytest-style testing; G-Eval (rubric-based), DAG (deterministic), ArenaGEval (pairwise), plus RAG and agent metrics. https://deepeval.com
8. Anthropic, *Demystifying Evals for AI Agents* (2026) and *Building Effective Agents* (2024). First-party agent-eval methodology: grader families, capability vs. regression evals, pass@k / pass^k, agent vs. eval harnesses, eval-driven development, and a framework appendix (Harbor, Braintrust, LangSmith, Langfuse, Phoenix). https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents
9. LangChain, *LangSmith Evaluation*, plus the open-source *OpenEvals* and *AgentEvals* (trajectory match + trajectory LLM-as-judge; LangGraph graph trajectory). https://docs.langchain.com/langsmith/evaluation · https://github.com/langchain-ai/agentevals
10. Microsoft / AutoGen, *AutoGenBench* (CLI benchmark runner) and *AgentEval* (Critic / Quantifier / Verifier agents); AutoGen is now succeeded by the Microsoft Agent Framework, with production evaluation via the Azure AI Foundry evaluation SDK.
11. OpenAI, *Evals API*, *trace grading*, and the *Agents SDK / AgentKit*; programmable graders also drive Reinforcement Fine-Tuning. https://developers.openai.com/api/docs/guides/agent-evals
12. Agent benchmarks: SWE-bench / SWE-bench Verified (Jimenez et al.), Terminal-Bench 2.0 (Harbor / Terminus-2), τ-bench / τ²-bench (Yao et al.; Sierra), BFCL (Patil, Yan et al.), ToolBench / ToolLLM (Qin et al.), GAIA (Mialon et al.), AgentBench (Liu et al.), WebArena / VisualWebArena (Zhou et al.; Koh et al.), OSWorld (Xie et al.), BrowseComp.
13. Eval / observability platforms: Harbor, Braintrust (`autoevals`), Langfuse, Arize Phoenix & AX, Comet Opik, Weights & Biases Weave, MLflow.
14. *Deep Agents* (LangChain) — the batteries-included agent harness built on LangGraph (planning, virtual filesystem, sub-agents, middleware, pluggable backends). Docs: https://docs.langchain.com/oss/python/deepagents/overview · Repo: https://github.com/langchain-ai/deepagents
15. LangChain, *Improving Deep Agents with Harness Engineering* (2026) — `agent = model + harness`; harness-only changes moved `deepagents-cli` +13.7 pts (52.8 → 66.5) on Terminal-Bench 2.0. https://www.langchain.com/blog/improving-deep-agents-with-harness-engineering
16. LangChain, *Evaluating Deep Agents* (2025) — per-case success criteria and trajectory/state assertions across four Deep Agents apps; curated skills lifted task completion from ~9% to ~82%.
17. *better-harness* — Deep Agents example: an outer Deep Agent optimizes an inner agent's harness surfaces against train/holdout/scorecard splits via pytest or Harbor runners. https://github.com/

langchain-ai/deepagents/tree/main/examples/better-harness. Inspired by karpathy's `autoresearch` and Stanford IRIS Lab's *Meta-Harness*.